WebdriverIO MCP

Automatización con MCP, WebdriverIO y Appium

Guía completa en español: qué es el servidor MCP de WebdriverIO, cómo levantar Appium y el emulador en macOS y Windows, cómo ejecutar la suite de pruebas y cómo interactuar con todo el stack desde opencode.

> Contenido traducido y resumido de webdriver.io/docs/mcp · Actualizado con el estado actual del proyecto

---

Contenido

#	Sección	Resumen
01	¿Qué es el servidor MCP?	Qué es `@wdio/mcp`, sus ventajas y su interfaz unificada
02	¿Qué puede hacer?	Capacidades de navegador y de apps móviles
03	Arquitectura del stack	IA ↔ MCP ↔ WebdriverIO ↔ Appium ↔ dispositivo
04	Instalación en opencode	Config global con el bloque `mcp` y variables de Appium
05	Herramientas disponibles	Tools actuales de navegador, móvil, Electron y cloud
06	Selectores	CSS, XPath, UiAutomator, predicados iOS y selectores de la app
07	Levantar emulador y Appium	Requisitos, variables de entorno y arranque en macOS y Windows
08	Ejecución de la suite	Scripts npm, comandos por sistema, filtros y Allure
09	Interactuar con el stack	Ejemplos de prompts y flujo completo con tu APK
10	Solución de problemas	Fallos típicos y qué revisar en cada plataforma
11	Recursos oficiales	Documentación, repositorio y paquetes npm

> **Estado actual del proyecto (macOS):** Node 20.18.1 · npm 10.8.2 · Appium 2.19.0 · uiautomator2 3.5.2 · emulador `emulator-5554`. La suite Mocha tiene **31 tests en 5 specs** (login 9, forms 9, swipe 5, drag 5, home 3), todos en verde. En Windows, `wdio.conf.ts` detecta el SDK y la suite arranca el emulador y Appium automáticamente.

---

01 · ¿Qué es el MCP de WebdriverIO?

Es un servidor **Model Context Protocol (MCP)** (`@wdio/mcp`) que permite a asistentes de IA (opencode, Claude Desktop, Claude Code) automatizar navegadores web y aplicaciones móviles nativas.

Mobile-First

- A diferencia de MCP servers solo para navegador, soporta iOS y Android nativos mediante Appium.

- Apps híbridas: cambio de contexto nativo ↔ WebView.

Selectores multiplataforma

- Detección inteligente de elementos: genera varias estrategias a la vez (Accessibility ID, XPath, UiAutomator, predicados iOS).

Ecosistema WebdriverIO

- Construido sobre el probadísimo framework WebdriverIO y su ecosistema de servicios y reporters.

Interfaz unificada

- 🖥️ Navegadores de escritorio (Chrome con o sin headless).
- 📱 Apps móviles nativas (simulador iOS / emulador Android / dispositivos reales vía Appium).
- 📳 Apps híbridas (nativo + WebView).

02 · ¿Qué puede hacer?

Navegador (Chrome)

- Lanzar Chrome en modo con ventana o headless, con dimensiones personalizadas y navegación inicial.
- Navegar, hacer clic, escribir, hacer scroll.
- Analizar la página: árbol de accesibilidad y elementos visibles (con paginación y filtros).
- Capturas de pantalla optimizadas (redimensionadas y comprimidas a máx. 1 MB).
- Gestión de cookies (obtener, crear, borrar).
- Ejecutar JavaScript en el contexto del navegador.

Apps móviles (iOS/Android)

- Ciclo de vida de sesión y de la app (estado, activar/terminar).
- Gestos táctiles: tap, swipe, drag & drop.
- Detección de elementos con múltiples estrategias y paginación.
- Cambio de contexto en apps híbridas.
- Control del dispositivo: rotación, teclado, geolocalización.
- Permisos y alertas automáticas.
- Comandos móviles de Appium (pressKey, deepLink, shell…).

> **Ojo con el contexto:** usar un servidor MCP suma tokens al contexto. Conviene ser selectivo con los MCPs habilitados; servidores grandes (p. ej. el de GitHub) pueden consumir mucho.

03 · Arquitectura: el stack completo

El servidor MCP actúa de puente entre el asistente de IA y los navegadores o dispositivos móviles:

```
Asistente IA → @wdio/mcp → WebDriverIO → Appium → Dispositivo
    ↑            ↑            ↑          ↑      ↑
  opencode   Servidor MCP  Cliente    Puerto   Chrome · iOS ·
             · stdio      WebDriver    4723     Android
```

Gestión de sesiones

- Modelo de sesión única: solo un navegador o app activa a la vez.
- El estado de la sesión se mantiene globalmente entre llamadas.
- Auto-detach: las sesiones con estado preservado (`noReset: true`) se desvinculan solas al cerrar.

Detección de elementos

- Web: script optimizado que encuentra los elementos visibles e interactuables (selectores CSS, IDs, clases, ARIA). Filtra por viewport por defecto.
- Móvil: parseo XML eficiente de la página (2 llamadas HTTP frente a 600+), con clasificación por plataforma.

Estrategias de localización generadas por elemento (móvil)

Estrategia	Descripción
Accessibility ID	Multiplataforma, la más estable (el recomendado por WebdriverIO)
Resource ID / Name	Atributo nativo de Android / iOS
Texto / Label	Coincidencia por texto visible o etiqueta
XPath	XPath completo y simplificado
UiAutomator / Predicates	Selectores nativos de Android (UiAutomator) e iOS (Predicates)

> **Recomendación oficial:** usa siempre que puedas Accessibility ID (`~loginButton`): es la estrategia multiplataforma más estable y se usa tanto en iOS como en Android.

04 · Instalación y configuración en opencode

La forma más sencilla es usar `npx` sin instalación local:

```
npx @wdio/mcp
```

Para que opencode lo cargue en todos tus proyectos, agrégalo al archivo de configuración global:

- **macOS/Linux:** `~/.config/opencode/opencode.jsonc`
- **Windows:** `%USERPROFILE%\.config\opencode\opencode.jsonc`

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "wdio-mcp": {
      "type": "local",
      "command": ["npx", "-y", "@wdio/mcp"],
      "enabled": true
    }
  }
}
```

> **Importante:** tras guardar la configuración hay que cerrar y reabrir opencode. Pon `enabled: false` para desactivarlo temporalmente sin borrarlo.

Verifica que el servidor quedó conectado (mismo comando en macOS y Windows):

```
opencode mcp list
```

Variables de entorno de Appium

El servidor MCP configura su conexión a Appium con estas variables:

Variable	Descripción	Default
`APPIUM_URL`	Host de Appium	`127.0.0.1`
`APPIUM_URL_PORT`	Puerto	`4723`
`APPIUM_PATH`	Ruta base	`/`

> **Optimización y robustez:** el servidor usa formato TOON (notación orientada a tokens) para comunicaciones ligeras, comprime las capturas a máx. 1 MB con Sharp, filtra por viewport y devuelve los errores como texto sin perder el estado de la sesión.

TypeScript: el paquete está escrito en TS con tipos completos, así que también puede integrarse y extenderse programáticamente con autocompletado y seguridad de tipos.

Versión de Node: la suite funciona con Node 20.18.1. Para `@wdio/mcp` y Allure 3 se recomienda **Node 22 LTS** (Allure advierte si usas una versión menor a 20.19).

05 · Herramientas disponibles

El servidor actual expone herramientas para navegador, móvil, Electron, mocking, extensiones y cloud. Estas son las principales:

Sesión y navegador

Herramienta	Descripción
`launch_chrome`	Lanza Chrome con depuración remota (perfil temporal)
`start_session`	Inicia una sesión de navegador, Electron o app móvil
`attach_session`	Se adjunta a una sesión WebDriver/Appium existente
`close_session`	Cierra o desvincula la sesión
`navigate`	Navega a una URL
`get_tabs` · `switch_tab`	Lista y cambia de pestaña
`click_element`	Hace clic en un elemento (navegador)
`set_value`	Limpia y escribe texto en un campo
`get_elements`	Elementos interactuables (con filtros y paginación)
`get_accessibility_tree`	Árbol de accesibilidad (con filtros por rol)
`get_screenshot`	Captura optimizada automáticamente
`scroll`	Hace scroll vertical (navegador)
`get_cookies` · `set_cookie` · `delete_cookies`	Gestión de cookies
`execute_script`	Ejecuta JavaScript en la página
`switch_frame`	Entra o sale de un iframe
`emulate_device`	Emula un dispositivo móvil en Chrome

Móvil (iOS / Android)

Herramienta	Descripción
`start_session`	Inicia la app con `platform: "android"` o `"ios"`
`attach_session`	Se adjunta a una sesión Appium existente
`tap_element`	Toca un elemento o coordenadas (recomendado en iOS)
`swipe`	Desliza en una dirección
`drag_and_drop`	Arrastra y suelta entre ubicaciones
`get_app_state`	Comprueba si la app está instalada/en ejecución
`get_contexts` · `switch_context`	Cambio de contexto en apps híbridas (nativo ↔ WebView)
`rotate_device`	Rota a vertical/horizontal
`set_geolocation`	Fija las coordenadas GPS
`hide_keyboard`	Oculta el teclado en pantalla
`execute_script`	Comandos móviles de Appium (`mobile: ...`)

Extras

Herramienta	Descripción
`upload_app` · `list_apps`	Apps en proveedores cloud (BrowserStack, Sauce Labs, etc.)
`mock` · `get_mock_calls` · `manage_mock`	Mock de red (BiDi) o de APIs de Electron
`open_web_extension`	Carga una extensión web y abre sus páginas
`execute_electron_script` · `trigger_electron_deeplink`	Automatización de Electron
`query_docs`	Busca en la documentación oficial de WebdriverIO

> **Nombres actuales:** `start_session` (antes `start_app_session`), `get_elements` (antes `get_visible_elements`), `get_accessibility_tree` (antes `get_accessibility`), `get_screenshot` (antes `take_screenshot`) y `launch_chrome` (antes `start_browser`).

06 · Sintaxis de selectores

El servidor soporta múltiples estrategias de localización, tanto en web como en móvil.

Web · CSS / XPath / Texto

CSS

```
button.my-class
#element-id
[data-testid="login"]
```

XPath

```
//button[@class='submit']
//a[contains(text(), 'Click')]
```

Texto (WebdriverIO)

```
button=Exact Button Text
a*=Partial Link Text
```

> **Recomendado:** en web, usa `data-testid` cuando exista; en móvil, prioriza el Accessibility ID.

Móvil · estrategias multiplataforma

Accessibility ID (recomendado) — funciona en iOS y Android:

```
~loginButton
```

Android UiAutomator

```
android=new UiSelector()
.text("Login")
```

iOS Predicate String

```
-ios predicate string:label = "Login"
```

iOS Class Chain

```
-ios class chain:**/XCUIElementTypeButton[`label = "Login"`]
```

XPath (ambas plataformas)

```
//android.widget.Button[@text="Login"]
//XCUIElementTypeButton[@label="Login"]
```

> «~» = **Accessibility ID:** la estrategia más estable y la que genera el servidor por defecto junto con las demás.

Selectores de la app de ejemplo (`com.wdiodemoapp`)

Pantalla	Selectores
Login	`~Login-screen` , `~input-email` , `~input-password` , `~button-LOGIN` , `~button-sign-up-container`
Forms	`~Forms-screen` , `~text-input` , `~input-text-result` , `~switch` , `~switch-text` , `~Dropdown`
Swipe	`~Swipe-screen` , `~card`
Drag	`~Drag-drop-screen` , `~drag-l1` … `~drag-r3` , `~drop-l1` … `~drop-r3` , `~renew` , `~button-Retry`
Home	`~Home-screen`
Bottom bar	`~Home` , `~Webview` , `~Login` , `~Forms` , `~Swipe` , `~Drag` , `~Menu`

> En la app actual **no existe** `~Carousel` : el carrusel se localiza con `~Swipe-screen` y cada tarjeta con `~card` . En Forms, las opciones del dropdown son `CheckedTextView` .

07 · Levantar el emulador y Appium (macOS y Windows)

La suite del proyecto ya automatiza el arranque: `npm run test:*` detecta el SDK, inicia el emulador si no hay ningún dispositivo conectado y levanta Appium con `@wdio/appium-service` . Los pasos manuales de esta sección son para usar el MCP de forma interactiva o para diagnosticar problemas.

Requisitos

Componente	Versión / nota	Verificación
Node.js	20.x (probado con 20.18.1; 22 LTS recomendado)	`node -v`

Componente	Versión / nota	Verificación
npm	10.x	`npm -v`
JDK	17 recomendado	`java -version`
Android Studio + SDK	Platform-Tools, Emulator y al menos un AVD	`adb devices`
Appium	2.19.0	`appium --version`
Driver	uiautomator2 3.5.2	`appium driver list --installed`

> iOS requiere macOS + Xcode. En Windows solo está disponible Android.

7.1 Variables de entorno

macOS (zsh/bash)

```
export ANDROID_HOME="$HOME/Library/Android/sdk"
export ANDROID_SDK_ROOT="$ANDROID_HOME"
export PATH="$ANDROID_HOME/platform-tools:$ANDROID_HOME/emulator:$ANDROID_HOME/cmdline-tools/latest/bin:$PATH"
```

Para hacerlo permanente, agrega esas líneas a `~/.zshrc` (o `~/.bash_profile`) y ejecuta `source ~/.zshrc`.

Windows PowerShell

```
$env:ANDROID_HOME = "$env:LOCALAPPDATA\Android\Sdk"
$env:ANDROID_SDK_ROOT = $env:ANDROID_HOME
$env:Path += ";$env:ANDROID_HOME\platform-tools;$env:ANDROID_HOME\emulator;$env:ANDROID_HOME\cmdline-tools\latest\bin"
```

Para hacerlo permanente: System Properties → Environment Variables, o `setx ANDROID_HOME "%LOCALAPPDATA%\Android\Sdk"` y agrega las rutas al `Path`.

Windows CMD

```
set ANDROID_HOME=%LOCALAPPDATA%\Android\Sdk
set ANDROID_SDK_ROOT=%ANDROID_HOME%
set PATH=%PATH%;%ANDROID_HOME%\platform-tools;%ANDROID_HOME%\emulator;%ANDROID_HOME%\cmdline-tools\latest\bin
```

> `wdio.conf.ts` también detecta el SDK automáticamente en `%LOCALAPPDATA%\Android\Sdk` y `%USERPROFILE%\AppData\Local\Android\Sdk`, así que las variables son un respaldo.

7.2 Instalar Appium y el driver (una sola vez)

```
npm install -g appium
appium driver install uiautomator2
```

También puedes usar la instalación local del proyecto:

```
npx appium driver install uiautomator2
```

Verifica en macOS o Windows:

```
appium --version
appium driver list --installed
```

7.3 Arrancar el emulador

Automático (recomendado, macOS y Windows):

```
npm run emulator
```

Arranca el AVD indicado en `ANDROID_AVD` (o el primero disponible) y espera a que `sys.boot_completed` sea `1`.

Manual:

```
# macOS
emulator -avd Medium_Phone_API_36.1
```

```
# Windows PowerShell
& "$env:LOCALAPPDATA\Android\Sdk\emulator\emulator.exe" -avd Medium_Phone_API_36.1
```

```
REM Windows CMD
%LOCALAPPDATA%\Android\Sdk\emulator\emulator.exe -avd Medium_Phone_API_36.1
```

Verifica en ambos sistemas:

```
adb devices
# → emulator-5554    device
```

7.4 Arrancar Appium

Para el MCP (interactivo), en una terminal aparte:

```
appium
# escucha en 127.0.0.1:4723
```

En Windows es el mismo comando. Si usas la instalación local: `npx appium`.

> Para la suite Mocha **no** hace falta arrancar Appium a mano: `services: ['appium']` lo inicia y lo detiene en cada corrida. El servicio elige el puerto 4723 si está libre (o uno libre) y lo inyecta en las capabilities.

7.5 iOS (solo macOS)

1. Instalar Xcode y las Command Line Tools:

   ```
   xcode-select --install
   ```

2. Instalar Appium y el driver XCUITest:

   ```
   npm install -g appium
   appium driver install xcuitest
   ```

3. Crear/ver simuladores y arrancar Appium:

   Xcode → Window → Devices and Simulators. Verifica con `xcrun simctl list devices`. Para dispositivos reales necesitas el UDID (40 caracteres). Luego ejecuta `appium`.

7.6 Verificación rápida

```
node -v
npm -v
appium --version
appium driver list --installed
adb devices
emulator -list-avds
```

> En Windows, si `adb` y `emulator` no están en el PATH, usa `%LOCALAPPDATA%\Android\Sdk\platform-tools\adb.exe` y `%LOCALAPPDATA%\Android\Sdk\emulator\emulator.exe`, o define el PATH como en 7.1.

08 · Ejecución de la suite (macOS y Windows)

Todos los comandos se ejecutan desde la raíz del proyecto. Los scripts npm son idénticos en macOS y Windows; solo cambian las variables de entorno y la sintaxis del shell.

8.1 Instalación inicial

```
npm install
npx appium driver install uiautomator2
```

> `DriverIO.apk` debe estar en la raíz del proyecto. Está en `.gitignore`, así que no viaja en el clon; si falta, `onPrepare` falla con un mensaje explícito.

8.2 Scripts npm

Comando	Qué hace
`npm run emulator`	Arranca el emulador (si no hay uno conectado) y espera el boot
`npm run wdio`	Ejecuta todos los `.spec.ts`
`npm run test`	Ejecuta `login.spec.ts`
`npm run test:login`	Ejecuta `login.spec.ts`
`npm run test:forms`	Ejecuta `forms.spec.ts`
`npm run test:swipe`	Ejecuta `swipe.spec.ts`
`npm run test:drag`	Ejecuta `drag.spec.ts`
`npm run test:home`	Ejecuta `home.spec.ts`
`npm run test:smoke`	Ejecuta solo las validaciones visuales (grep)
`npm run test:all`	Ejecuta toda la suite

> Todos los `test:*` arrancan el emulador automáticamente si no hay un dispositivo conectado, y Appium se inicia solo con el servicio.

8.3 macOS (zsh/bash)

```
# Suite completa: arranca emulador y Appium automáticamente
npm run test:all

# Por feature
npm run test:login
npm run test:forms
npm run test:swipe
npm run test:drag
npm run test:home

# Solo validaciones visuales (smoke)
npm run test:smoke

# Un spec puntual
npx wdio run ./wdio.conf.ts --spec ./features/specs/login.spec.ts

# Filtrar por ID de caso de prueba
npx wdio run ./wdio.conf.ts --mochaOpts.grep "CP-019"

# Excluir por nombre
npx wdio run ./wdio.conf.ts --mochaOpts.grep "Arrastrar" --mochaOpts.invert
```

8.4 Windows PowerShell

```
# Suite completa: arranca emulador y Appium automáticamente
npm run test:all

# Por feature
npm run test:login
npm run test:forms
npm run test:swipe
npm run test:drag
npm run test:home

# Solo validaciones visuales (smoke)
npm run test:smoke

# Un spec puntual
npx wdio run ./wdio.conf.ts --spec ./features/specs/login.spec.ts

# Filtrar por ID de caso de prueba
npx wdio run ./wdio.conf.ts --mochaOpts.grep "CP-019"
```

> En Windows CMD los comandos son los mismos; para variables de entorno reemplaza `$env:VAR = "valor";` por `set VAR=valor &&`. Si `npx` pide confirmación, agrega `--yes`.

8.5 Variables de entorno útiles

Variable	Default	macOS/Linux	Windows PowerShell
`ANDROID_AVD`	primer AVD	`ANDROID_AVD=Medium_Phone_API_36.1 npm run test:all`	`$env:ANDROID_AVD="Medium_Phone_API_36.1"; npm run test:all`
`EMULATOR_BOOT_TIMEOUT`	`240000`	`EMULATOR_BOOT_TIMEOUT=360000 npm run test:all`	`$env:EMULATOR_BOOT_TIMEOUT="360000"; npm run test:all`
`ANDROID_DEVICE_NAME`	serial detectado	`ANDROID_DEVICE_NAME=emulator-5554 npm run test:all`	`$env:ANDROID_DEVICE_NAME="emulator-5554"; npm run test:all`
`ANDROID_PLATFORM_VERSION`	auto	`ANDROID_PLATFORM_VERSION=14 npm run test:all`	`$env:ANDROID_PLATFORM_VERSION="14"; npm run test:all`
`ALLURE_REPORT`	desactivado	`ALLURE_REPORT=1 npm run test:all`	`$env:ALLURE_REPORT="1"; npm run test:all`

> `ANDROID_DEVICE_NAME` fija el `appium:deviceName` de las capabilities; el emulador se sigue garantizando con `ensureEmulatorRunning()`. `APPIUM_HOST` y `APPIUM_PORT` existen como respaldo si usas un Appium externo.

8.6 Reportes Allure

macOS/Linux:

```
ALLURE_REPORT=1 npm run test:all
npx allure serve allure-results
npx allure generate allure-results -o allure-report --clean
```

Windows PowerShell:

```
$env:ALLURE_REPORT = "1"
npm run test:all
Remove-Item Env:ALLURE_REPORT

npx allure serve allure-results
npx allure generate allure-results -o allure-report --clean
```

Windows CMD:

```
set ALLURE_REPORT=1 && npm run test:all
```

> `allure-results/` y `allure-report/` están en `.gitignore`. Última corrida registrada: **31/31 passed**.

8.7 Notas específicas de Windows

- `wdio.conf.ts` busca el SDK en `%LOCALAPPDATA%\Android\Sdk`, `%USERPROFILE%\AppData\Local\Android\Sdk`, `ANDROID_HOME` y `ANDROID_SDK_ROOT`.
- Usa `adb.exe` y `emulator.exe`; si no están en el PATH, defínelos como en 7.1.
- Para rutas con espacios, usa comillas: `cd "C:\Repositorios\web Driver IO"`.
- El APK debe estar en la raíz del proyecto; el archivo está ignorado por Git.

09 · Cómo interactuar con todo el stack

Con el MCP activo, el asistente usa las tools automáticamente: solo tienes que pedirle lo que quieres lograr en lenguaje natural.

Ejemplos · Navegador

- "Abre Chrome y navega a https://webdriver.io"
- "Haz clic en el botón 'Get Started'"
- "Toma un screenshot de la página"
- "Encuentra todos los enlaces visibles"

Ejemplos · Móvil

- "Inicia mi app iOS en el simulador iPhone 15"
- "Toca el botón de login"

- “Desliza hacia arriba para hacer scroll”
- “Toma un screenshot de la pantalla actual”

Flujo completo de ejemplo (Android · con tu proyecto)

1. Prepara el entorno
   - **Opción suite:** `npm run test:all` levanta emulador y Appium automáticamente.
   - **Opción MCP:** `npm run emulator` y, en otra terminal, `appium` (127.0.0.1:4723).
2. Pide iniciar la sesión con tu APK

   En la raíz de webDriverIOMobile tienes `DriverIO.apk`. Pídele: “Inicia la app `DriverIO.apk` en el emulador Android”. El MCP llama a `start_session` con la ruta de la APK.
3. Interactúa
   - “Toca el botón de login” → `tap_element`
   - “Escribe 'demo' en el campo de usuario” → `set_value`
   - “Haz swipe hacia arriba” → `swipe`
4. Analiza y captura evidencia
   - “Toma un screenshot” → `get_screenshot` (comprimido a máx. 1 MB)
   - “Muéstrame el árbol de accesibilidad” → `get_accessibility_tree`

Trucos y buenas prácticas

Práctica	Detalle
Sesión única	Solo puede haber un navegador o app activa. Cierra la sesión (`close_session`) antes de cambiar de blanco
Prioriza el Accessibility ID	Selectores `~id` que no dependen del layout son los más robustos multiplataforma
Contextos híbridos	En apps nativo+WebView: `get_contexts` lista los contextos y `switch_context` cambia entre nativo y webview
Primera ejecución	La descarga de `npx @wdio/mcp` puede superar los 30 s de timeout. Precalienta con `npx -y @wdio/mcp` una vez
Appium para MCP	La suite lo levanta sola; para el MCP arranca `appium` manualmente en 4723

> Los errores nunca rompen la sesión: el servidor MCP devuelve los errores como contenido de texto y preserva el estado de la sesión, así que puedes encadenar intentos sin reiniciar.

10 · Solución de problemas

Síntoma	Qué revisar
El navegador no arranca	Chrome instalado · puerto de depuración 9222 libre · probar modo headless si hay problemas de pantalla

Síntoma	Qué revisar
Conexión Appium fallida	Servidor appium corriendo · `APPIUM_URL` y puerto correctos · driver instalado (`appium driver list`)
Problemas con simulador iOS	Xcode actualizado · simuladores disponibles (`xcrun simctl list devices`) · UDID correcto en dispositivos reales
Problemas con emulador Android	`ANDROID_HOME` configurado · emulador en marcha (`adb devices`)
MCP “failed / timeout”	Primera descarga de `npx @wdio/mcp` : precargar la caché y reintentar · Node 20+ (22 LTS recomendado)

El emulador deja de responder tras varias corridas

- **Síntoma:** timeouts, `invalid session id` , `A session is either terminated or not started` .
- **Solución:** ciérralo y vuelve a arrancarlo:

```
adb emu kill
npm run emulator
```

APK no encontrado

- **Causa:** `DriverIO.apk` está en `.gitignore` y no viaja en el clon.
- **Solución:** colócalo en la raíz del proyecto. Sin él, `onPrepare` falla con un mensaje explícito.

Windows no detecta el SDK

- **Causa:** `ANDROID_HOME` / `ANDROID_SDK_ROOT` no definidos y el SDK no está en la ruta esperada.
- **Solución:** define las variables como en 7.1. `wdio.conf.ts` también busca en `%LOCALAPPDATA%\Android\Sdk` y `%USERPROFILE%\AppData\Local\Android\Sdk` .

Advertencia de engine al usar Allure

- **Síntoma:** `entities@8.1.0` requiere Node `>=20.19.0` .
- **Solución:** la suite funciona con Node 20.18.1; para evitar la advertencia usa Node 22 LTS.

`~Carousel` no encontrado

- **Causa:** la app actual no expone ese accessibility id.
- **Solución:** usa `~Swipe-screen` para la pantalla y `~card` para las tarjetas.

El alert de login bloquea la bottom bar

- **Causa:** tras un login exitoso aparece el alert nativo “Success / You are logged in!”.
- **Solución:** pulsa el botón `OK` (o usa el helper `prepareForNavigation()` de la suite) antes de navegar.

El teclado tapa la bottom bar en Forms

- **Solución:** oculta el teclado (`hide_keyboard` en MCP o `driver.hideKeyboard()` en la suite) antes de tocar la bottom bar o el dropdown.

El modal “Congratulations” bloquea Renew en Drag

- **Causa:** al resolver el puzzle aparece un modal con overlay que bloquea `~renew` .

- **Solución:** pulsa `~button-Retry` para reiniciar; si el puzzle no está resuelto, `~renew` funciona normalmente.

Appium no arranca o el puerto está ocupado

- **Solución:** verifica `appium --version` y `appium driver list --installed`; libera el puerto 4723 o deja que `@wdio/appium-service` elija uno libre. Para MCP, `APPIUM_URL_PORT` debe coincidir con el Appium en ejecución.

11 · Recursos oficiales

Documentación

- MCP WebdriverIO: https://webdriver.io/docs/mcp
- Configuración: https://webdriver.io/docs/mcp/configuration
- Transport y cloud: https://webdriver.io/docs/mcp/transport
- Especificación MCP: https://modelcontextprotocol.io

Código y paquetes

- Paquete npm: `@wdio/mcp`
- Repo GitHub: https://github.com/webdriverio/mcp
- Herramientas y selectores: https://webdriver.io/docs/mcp/tools · https://webdriver.io/docs/mcp/selectors

---

> *Créditos y atribución: contenido traducido y resumido de la documentación oficial de WebdriverIO MCP (webdriver.io/docs/mcp, © OpenJS Foundation y colaboradores de WebdriverIO 2026). Iconos: Twemoji (© 2020 Twitter, Inc.), licencia CC-BY 4.0 · ilustraciones y diseño generados con opencode.*